HybridSQL: Ambiguity-Aware Natural Language Query Processing over Structured and Unstructured Sources

Grishma Bellani*
IIIT Delhi
grishma22189@iiitd.ac.in

Harsh Rajput*
IIIT Delhi
harsh22201@iiitd.ac.in

Mukesh Mohania
IIIT Delhi
mukesh@iiitd.ac.in

ABSTRACT

Recent advances in large language models (LLMs) have substantially improved the effectiveness of natural language-to-SQL (NL2SQL) systems. However, in practical settings users often have limited awareness of the underlying database schema, including what information is stored and how it is represented. Consequently, their queries are frequently expressed using language that differs from the database vocabulary, resulting in attribute and value level mismatches between user expressions and database representations. Such queries may also require information from unstructured sources beyond what is explicitly captured in structured tables. Moreover, natural language queries are often ambiguous and admit multiple plausible interpretations, which can lead to fundamentally different query outcomes. These challenges expose key limitations of existing NL2SQL systems, which struggle to robustly handle intent ambiguity, schema–vocabulary misalignment, and the closed-world assumption of relational query processing that restricts answers to structured data alone.

We demonstrate **HybridSQL**, an interactive NL interface that addresses all these challenges within a unified pipeline. (i) HybridSQL detects and resolves intent-level ambiguity through a user clarification loop leveraging insights from prior query analyses. (ii) It further mitigates schema misalignment by grounding user terminology against actual schema attribute(s) and stored value(s) via embedding-based candidate retrieval. For queries that require knowledge beyond the database, (iii) HybridSQL decomposes execution into structured SQL queries and unstructured LLM-based queries, enabling coherent information integration across structured and unstructured sources.

1 INTRODUCTION

The relational model and SQL remain the dominant paradigm for data management, forming the backbone of modern analytical systems. Recent advances in large language models (LLMs) have further improved data accessibility by enabling natural language interfaces to databases (NL2SQL) [3, 6]. Allowing users to express analytical questions without requiring knowledge of schema structures or query syntax. Despite this progress, the practical deployment of NL interfaces still faces fundamental challenges that limit their effectiveness in real-world analytical settings.

Intent Ambiguity. Natural language queries often admit multiple valid interpretations, particularly when evaluative, aggregative, or referential expressions such as popular, performing poorly, or recently, etc. are used [1, 5, 7]. Different interpretations may lead to different SQL formulations and query outcomes. Existing approaches to ambiguity resolution follow two broad strategies: Using LLMs to automatically generate candidate interpretations and disambiguate through subsequent selection, or Engaging users for clarification after ambiguity identification [2]. However, both strategies face important limitations. Automated approaches that resolve ambiguity without user involvement risk generating interpretations misaligned with actual intent. Conversely, approaches that enforce per-query clarification overlook accumulated ambiguity knowledge and user preference patterns.

Schema and Value misalignment. Another related ambiguity is the schema and value misalignment between the user vocabulary and the database representation [4]. Users frequently express concepts using terminology that does not directly correspond to canonical attribute names or stored values in the relational database. Relevant information may be encoded among multiple candidate attributes. Even when the correct attribute(s) is identified, the specific value(s) expressed in the user's terminology may differ from the representation stored in the database [5]. Therefore, LLM-driven NL2SQL systems can hallucinate unsupported value predicates during query synthesis, leading to incorrect query execution and misleading outcomes. Prior approaches typically address these issues along a single dimension—either selecting the correct attribute or resolving the corresponding stored value—but do not jointly handle both forms of misalignment in an integrated manner.

Closed world Assumption of Databases. Finally, NL2SQL systems are fundamentally constrained by the limitations of SQL itself. While SQL is highly effective for retrieving and aggregating explicitly stored structured facts, many real-world analytical questions require contextual interpretation, explanatory reasoning, or access to unstructured knowledge that lies beyond database schemas [8]. Recent hybrid querying efforts, such as SWAN [10], demonstrate the feasibility of addressing such beyond-database information needs by integrating relational query execution with LLM-based reasoning capabilities. However, these approaches typically rely on manually specified hybrid query formulations or predefined schema expansion strategies, rather than automatically determining which components of a natural language query require structured data retrieval versus unstructured semantic reasoning.

Motivating Example: Consider the query ***Q1****: "Give all the best performing cosmetics products from environmentally friendly brands"* posed over the schema `products(product_id, product_name, category, sub_category, sub_sub_category, brand, price)`. This query encapsulates all challenges discussed. (i) The phrase *"best performing"* introduces **intent ambiguity**, as it may correspond to several plausible performance indicators, such as high sales revenue, favorable customer ratings or reviews, or increased purchase frequency.

*Both authors contributed equally to this research.

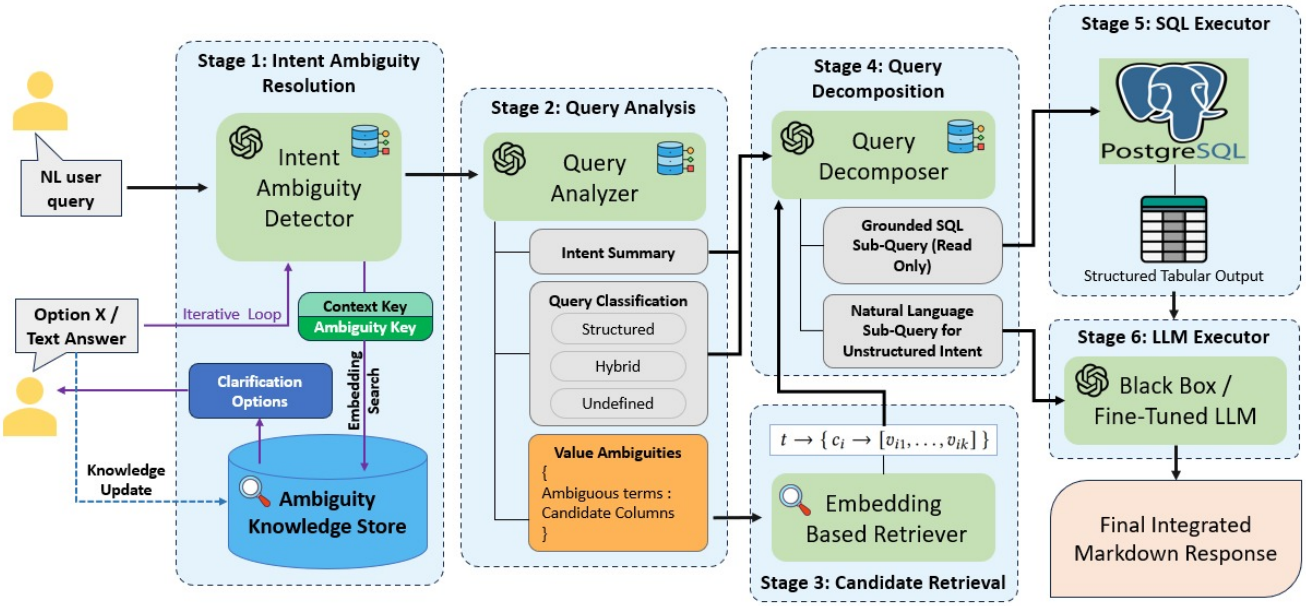


Figure 1: System Workflow

(ii) The expression *"cosmetics"* further illustrates **schema grounding complexities**, since product taxonomy can be represented at varying levels of granularity (e.g., category, sub-category, or sub-sub-category) and stored under a different term (e.g., makeup, beauty), (iii) While environmental-friendly/sustainability of a brand is not explicitly captured within structured schema fields and explanatory reasoning that extends beyond conventional SQL execution (e,g. companies transcript), requiring integration of contextual or unstructured knowledge sources. Queries of this composite analytical nature are increasingly prevalent in enterprise decision-support settings.

To address these challenges, we demonstrate **HybridSQL**, an interactive natural language interface that unifies intent disambiguation, schema grounding, and hybrid structured–unstructured query execution within a single end-to-end pipeline. Intent ambiguity is resolved through an iterative clarification mechanism that not only incorporates explicit user feedback but also leverages signals from prior query interactions and inferred user preferences maintained in an ambiguity knowledge store. HybridSQL also performs embedding-based schema grounding to automatically map user terminology to relevant database attributes and stored values, thereby reducing value hallucinations. For analytical queries that require knowledge beyond the structured schema, the system decomposes execution into : SQL queries over relational data and LLM-driven reasoning over unstructured sources. This integrated design enables the generation of coherent responses that combine structured evidence with unstructured information.

2 SYSTEM ARCHITECTURE

As illustrated in Figure 1, HybridSQL is designed as a *six-stage modular pipeline* that progressively transforms a natural language query into executable query over structured databases and unstructured knowledge sources. Each stage incrementally refines the query semantics, constraining and guiding subsequent processing steps to enable context-aware and grounded execution.

2.1 Stage 1: Intent Ambiguity Resolution

Given a user query and the database schema, the system first invokes an LLM-based ambiguity detector guided by a formally defined ambiguity taxonomy.

Ambiguity Taxonomy. We consider two primary ambiguity classes: (i) **decision-criterion ambiguity**, where subjective terms such as *best* may correspond to many optimization objectives (e.g., price, rating, or sales volume), and (ii) **reference ambiguity**, which captures uncertainty in temporal (e.g., *recently*), spatial (e.g., *nearby*), or attribute-level references (e.g., *loyal customers*). This taxonomy is inspired by AmbiSQL [2]; however, in HybridSQL, value and schema ambiguities are deliberately excluded at this stage to ensure that users are prompted to clarify only their conceptual goals.

Iterative Detection and Clarification. Ambiguity resolution proceeds through an interactive user refinement loop. After the ambiguity detection step, the system generates clarification questions to elicit the user's intended interpretation. For each identified ambiguity, users are presented with multiple candidate options to select from, along with the choice to provide custom clarification in free text. The clarification options presented to users take into account ambiguities observed in previous queries, along with the

corresponding user preference selections maintained in the ambiguity knowledge store. Once the user selects an option, the refined query context is fed back to the ambiguity detector, and the process repeats until no further ambiguities are identified.

Ambiguity Knowledge Store and Preference-Aware Ranking. Each detected ambiguity is represented as a (context_key, ambiguity_key) pair—e.g., ***Q1*** yields *context* = "cosmetic products" and *ambiguity* = "performance metric"—which together uniquely capture the nature of the ambiguity within a specific domain entity. When an ambiguity is encountered, the system performs embedding-based retrieval using *all-MiniLM-L12-v2* [9] to identify relevant prior ambiguity. Specifically, it first matches the *context_key* using a similarity threshold of $\tau_c = 0.85$, followed by a finer-grained match on the *ambiguity_key* within the retrieved contexts using a threshold of $\tau_a = 0.7$, thereby preserving strict semantic boundaries (e.g., distinguishing "best products" from "best customers"). If a relevant match is found, the system presents the top−3 most frequently selected clarification options, annotated with their historical selection percentages (e.g., *1. Sales Revenue (60%)*, *2. Reviews (15%)*, *3. Purchase Frequency (10%)*), thereby enabling informed selection based on prior user preferences. If no match is found, LLM-generated clarification options are presented and, upon user selection, stored in the knowledge store. This design ensures that clarification choices are progressively grounded in accumulated interaction history and promoting consistent intent specification across sessions.

2.2 Stage 2: Query Analysis

Following intent disambiguation, HybridSQL performs *semantic query analysis* to derive a structured representation of the user query.

Query Enrichment and Classification. The analyzer generates an *intent summary* capturing the fully disambiguated analytical objective and performs *query classification* into three categories: (i) **structured**, when the query can be fully expressed in SQL; (ii) **hybrid**, when answering requires both structured retrieval and unstructured reasoning; and (iii) **undefined**, when no part of the requested information can be extracted from structured information.

Note: The manner in which intent ambiguity is resolved in **Stage 1** has direct implications for query classification. For instance, if the notion of *"best performance"* is interpreted as *Sales Revenue*, the resulting analysis can be formulated over structured transactional attributes and executed using conventional SQL aggregation, and is thus classified as **structured**. In contrast, if the same expression is interpreted in terms of *Customer Reviews*, the required evidence may reside outside the structured schema and necessitate reasoning over unstructured textual information, and is therefore classified as **hybrid**.

Attribute-Value Ambiguity Identification. The analysis stage further detects *attribute- and value-level ambiguities* arising from misalignment between user vocabulary and database representation. For each ambiguous term, the analyzer proposes candidate attributes where semantic grounding should be attempted. These ambiguity annotations are forwarded to the candidate retrieval module for retrieving and ranking relevant attribute–value candidates for grounding.

2.3 Stage 3: Candidate Retrieval for Value Ambiguity Support

This stage supports downstream value ambiguity resolution by retrieving semantically relevant candidate values from the database. Unlike prior approaches that typically focus on a single dimension of uncertainty, HybridSQL explicitly surfaces signals for both **attribute relevance** and **value similarity**.

Given the set of candidate columns $\{c_1, c_2, \ldots\}$ identified in **Stage 2**, the system retrieves the set of `DISTINCT` stored values from each column. The ambiguous user term and the retrieved column values are embedded using a sentence-transformer model ("all-MiniLM-L6-v2), and semantic similarity search is performed independently for each candidate column. For every column, the top-k most similar values are retained as *candidate grounding signals*.

This process produces a *candidate map* of the form:

$$t \rightarrow \{\, c_i \rightarrow [v_{i1}, \ldots, v_{ik}] \,\},$$

where each ambiguous term t is associated with a ranked list of semantically similar candidate values for each candidate column c_i.

Importantly, Stage 3 performs *candidate retrieval only*. Final attribute selection and value grounding are deferred to the query decomposition stage, where the LLM is conditioned on these database-verified candidates during SQL generation.

2.4 Stage 4: Query Decomposition

This stage takes as input (i) the original natural language query, (ii) the enriched query representation from Analyzer (Stage 2), and (iii) the candidate attribute–value map resolved in Stage 3. The Decomposer partitions the query into two parts: (i) a SQL sub-query that retrieves the required structured information directly from relations, and (ii) an LLM-based unstructured sub-query that seeks additional reasoning or knowledge not derivable from structured schema fields.

Candidate attribute–value signals obtained in Stage 3 serve as grounding evidence during decomposition. The Decomposer selects the most semantically appropriate schema attributes and their resolved values to formulate precise SQL predicates, thereby aligning user vocabulary with the underlying database representation (e.g., "affordable cooking oil" → `category = 'Edible Oil'`).

The outputs of this stage are (i) a grounded, well-formed SQL query and (ii) a natural language prompt capturing the unstructured intent. Both outputs are subsequently forwarded to the Executor in Stage 5.

2.5 Stage 5: Structured Query Execution

In this stage, the SQL sub-query is executed on a PostgreSQL database. To ensure safety and prevent unintended modifications, the executor enforces strict read-only semantics by permitting only single-statement `SELECT` queries. The database returns results as a tabular data frame corresponding to the structured component of the user's analytical request.

2.6 Stage 6: Unstructured Integration

The structured results produced by SQL execution are provided as contextual evidence to the LLM along with the unstructured

sub-query. The LLM combines database-grounded evidence with semantic reasoning to generate a coherent final markdown response, which is then returned to the user.

Implementation Note. Currently, we employ an open-box LLM to handle unstructured reasoning and knowledge. Consequently, unstructured insights are derived directly from the model's pretrained weights. While the proposed architecture naturally supports extensions such as domain-adaptive LLM fine-tuning or retrieval-augmented generation (RAG) over curated unstructured corpora, incorporating these enhancements is beyond the scope of the present demonstration. The focus of this work is to validate the effectiveness of ambiguity-aware hybrid query decomposition and the integration of structured database evidence with LLM-based reasoning within a unified analytical pipeline.

3 EVALUATION

REFERENCES

[1] A. Bhaskar, T. Tomar, A. Sathe, and S. Sarawagi. Benchmarking and improving text-to-sql generation under ambiguity. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 7053–7074. Association for Computational Linguistics, 2023.

[2] Z. Ding, Y. Lin, and T. Zeng. Ambisql: Interactive ambiguity detection and resolution for text-to-sql. *arXiv preprint arXiv:2508.15276*, 2025.

[3] X. Dong, C. Zhang, Y. Ge, Y. Mao, J. Gao, J. Lin, and D. Lou. C3: Zero-shot text-to-sql with chatgpt. *arXiv preprint arXiv:2307.07306*, 2023.

[4] C. Dragusin, K. Mirylenka, C. Miksovic, M. Glass, N. Defosse, P. Scotton, and T. Gschwind. Grounding llms for database exploration: Intent scoping and paraphrasing for robust nl2sql. In *Proceedings of the VLDB 2025 Workshops (AIDB: Applied AI for Database Systems and Applications)*, London, United Kingdom, 2025.

[5] A. Floratou, F. Psallidas, F. Zhao, S. Deep, G. Hagleither, J. Cahoon, R. Alotaibi, J. Henkel, A. Singla, A. van Grootel, K. Deng, K. Lin, M. Campos, V. Emani, V. Pandit, W. Wang, and C. Curino. Nl2sql is a solved problem... not! In *Proceedings of the 14th Conference on Innovative Data Systems Research (CIDR)*, Chaminade, HI, USA, 2024.

[6] D. Gao, H. Wang, Y. Li, X. Sun, Y. Qian, B. Ding, and J. Zhou. Text-to-sql empowered by large language models: A benchmark evaluation. *arXiv preprint arXiv:2308.15363*, 2023.

[7] H. Lee, S. Iyer, M. Lapata, and Y. Artzi. Disambiguate first, parse later: Generating interpretations for ambiguity resolution in semantic parsing. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 10750–10767. Association for Computational Linguistics, 2023.

[8] H. Liu, S. J. Semnani, G. Campagna, and M. S. Lam. Suql: Conversational search over structured and unstructured data with function calling. In *Findings of the Association for Computational Linguistics: NAACL*, 2024.

[9] N. Reimers and I. Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of EMNLP-IJCNLP*, 2019.

[10] F. Zhao, D. Agrawal, and A. El Abbadi. Hybrid querying over relational databases and large language models. *arXiv preprint arXiv:2408.00884*, 2024.